

Biomedical Image-Analysis Pipeline: Stained-Nuclei Microscopy

A hybrid pipeline combining a local vision-language model, classical image processing, and a U-Net

Name : Deepak Kottakkal Murali

Student ID:24140337

Github ID: <https://github.com/dkmdeepak/Assignment-3-AI-imaging-Coding-Case-Study-and-Report/tree/main>

1. Methods overview

The dataset consists of 112 256×256 fluorescence images of stained nuclei belonging to four density categories (sparse, normal, dense, and clustered; ranging from 5 to 85 nuclei per image; 80/20/12 train/validation/test). The combination of three methods is used: (1) a local vision-language model (VLM) with Ollama providing an object-oriented schema-guided description of an image; (2) a classical procedure that uses Otsu thresholding, morphological processing, distance transform watershed for nuclei separation, and finally regionprops_table in order to compute per-object properties based on the mask of the nuclei; (3) a compact U-Net (encoder depth 3, number of channels from 8 to 64, skip connections, ~123k parameters) is used to predict the mask of nuclei directly from the image. At each step of the LLM, all the numerical features of the JSON record are computed in Python and then injected after the call.

2. Task 1 – Data preparation and VLM description

The images were gray-scaled and normalized to 256×256 dimensions. Figure (3b) and (3c) below show the sample panel and histogram of the intensities for each of the four density types showing that there is a clear bimodal intensity distribution between the nucleus pixels and the background pixels, meaning that there is a very low overlap between the two.

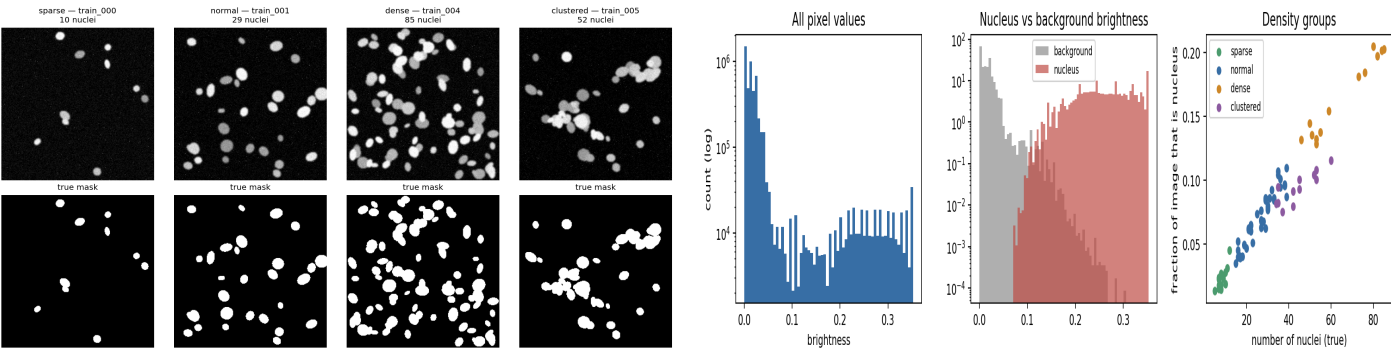


Fig. 1: image samples by density class with ground-truth masks (left); intensity histogram and density regime scatter plot (right).

2.1 Optimised prompt (structured, anchors the model as descriptive)

You are helping describe pictures from a microscopy teaching dataset. Only describe what you can actually see. You are not a doctor and this is not a diagnosis: do not name a disease or say anything about a patient. Answer with one JSON object only — no text before or after it. Schema: modality [fluorescence_microscopy|brightfield_microscopy|histology|radiograph|ct|mri|ultrasound|uncertain]; tissue_type [cell_nuclei|cell_culture|tissue_section|other|uncertain]; notable_features (2–5 short visual descriptions); image_quality [good|adequate|poor|uncertain]; confidence [high|medium|low]. If a property cannot be judged from pixels alone, answer "uncertain" — that is always acceptable. Do not count objects or give measurements.

Unstructured prompt ("What can you see in this medical image?") versus structured prompt, tested on a representative medical image using a local vision model (llava:7b via Ollama, llama3.2-vision could not handle image requests in this session):

Naive reply (free text, unconstrained)	Structured reply (schema-constrained JSON)
"From the image, it is a microscopic view of the cells... stained using the blue dye... and the circular objects could be nuclei... aligned in a lattice formation."	<pre>{"modality":"fluorescence_microscopy","tissue_type":"cell_culture","notable_features":["blue dots on dark background"],"image_quality":"good","confidence":"high"}</pre>

The naïve answer creates a clinical context ("tissue sample," "stages of cell division") that would be prohibited by the structured prompt rules. Testing the structured prompt three times at the same temperature (0.2) on the same image proves the phrasing is nondeterministic:

Run	tissue_type	notable_features	confidence
1	cell_culture	blue dots on dark background; dots vary in size; some dots touch	high
2	cell_nuclei	blue dots on dark background; dots vary in size; some dots touch	high
3	cell_culture	blue dots on dark background	high

All three responses were byte different, and tissue_type switched between cell_culture and cell_nuclei — which is precisely the kind of variability that a downstream CSV file can't handle, and explains why Task 2/4 always prevents model-derived numbers from being input unaltered.

3. Task 2 – Classical features and numbers-first interpretation

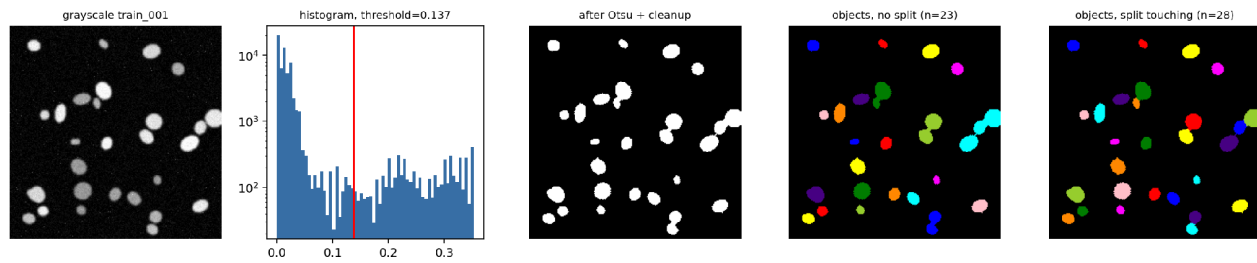


Fig. 2: Grayscale image, intensity histogram with Otsu threshold, clean binary mask, connected components, and finally the watershed segmentation of touching nuclei.

Optimised prompt (numbers only, model never sees the image)

PART 1: This is a scientific writing assistant. The measurement values provided by you pertain to a classical pipeline. You do not know what the image looks like. Please give your answer in two sections. PART 2: There are segmented objects in an image. The number of objects present is `n_objects`. This image has `density_class`, which could be sparse, moderate, dense, or uncertain. Shape regularity of these segmented objects is `shape_regularity`, which could be regular, mixed, irregular, or uncertain.

For the image `train_001` (with 28 objects adjacent to ground truth after watershed splitting), the measurement dict (`n_objects`, `area_fraction`, per-object area/eccentricity/solidity/circularity, along with the image-specific `contrast_score` and `focus_score` quality proxies) is serialised into the above prompt. Actual Ollama response:

"There are 28 segmented objects in this image. These objects cover a small portion of the image at only 8.69%. The objects are highly varied in size, ranging from a small size of 67 pixels all the way up to a larger size of 376 pixels, with a standard deviation of 91.8 pixels. The average object size is 203.5 pixels." → record: {"n_objects":28,"density_class":"sparse","shape_regularity":"irregular","quality_flag":"ok"}

Here, the validator found that one correction was needed as follows: the model had `density_class="sparse"`, but the calculation based on `area_fraction` gives "moderate" – the model's choice of words is retained as it is a matter of opinion, but the discrepancy is noted rather than being hidden. For `n_objects`, however, the calculated result always overrides the model's answer.

4. Task 3 – U-Net segmentation

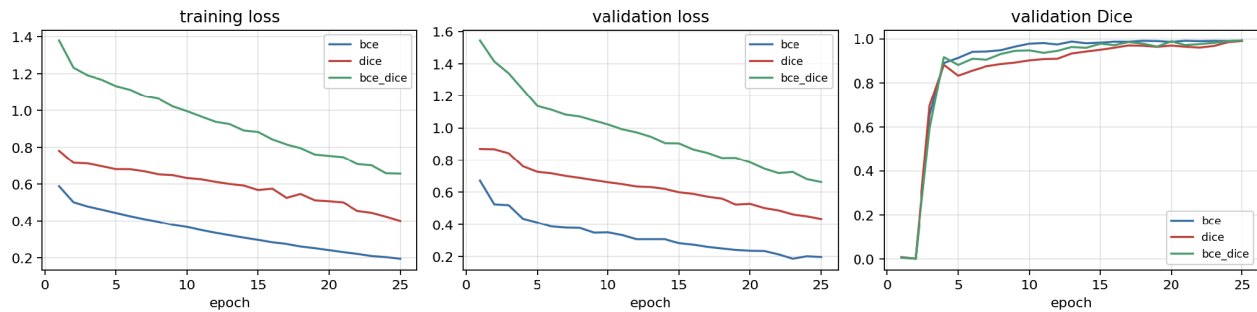


Fig. 3: training/validation loss and validation Dice for the three different loss functions (BCE, Soft Dice, BCE+Dice), 25 epochs each.

Method	Val Dice	Val IoU	Test Dice	Test IoU
Otsu + morphology	0.9742	0.9497	0.9749	0.9511
U-Net – BCE	0.9922	0.9846	—	—
U-Net – soft Dice	0.9905	0.9812	—	—
U-Net – BCE+Dice (best)	0.9927	0.9856	0.9918	0.9838

Validation was highest for BCE+Dice, and this model was used in all other tasks. The U-Net performed better than the Otsu baseline on each validation image (+0.0186 margin of Dice). The largest difference was seen on clustered/dense images (for example, `val_005`, clustered, 48 objects: U-Net 0.9922 vs Otsu 0.9606, margin 0.032), as a fixed intensity threshold would connect touching nuclei into one object, while the U-Net features are able to detect this boundary. Surprisingly, the smallest margin is a clustered image (`val_011`: 0.9950 vs 0.9825, margin 0.012). Density regime does not predict entirely the advantage of the U-Net on its own; it depends on how much the particular nuclei are touching.

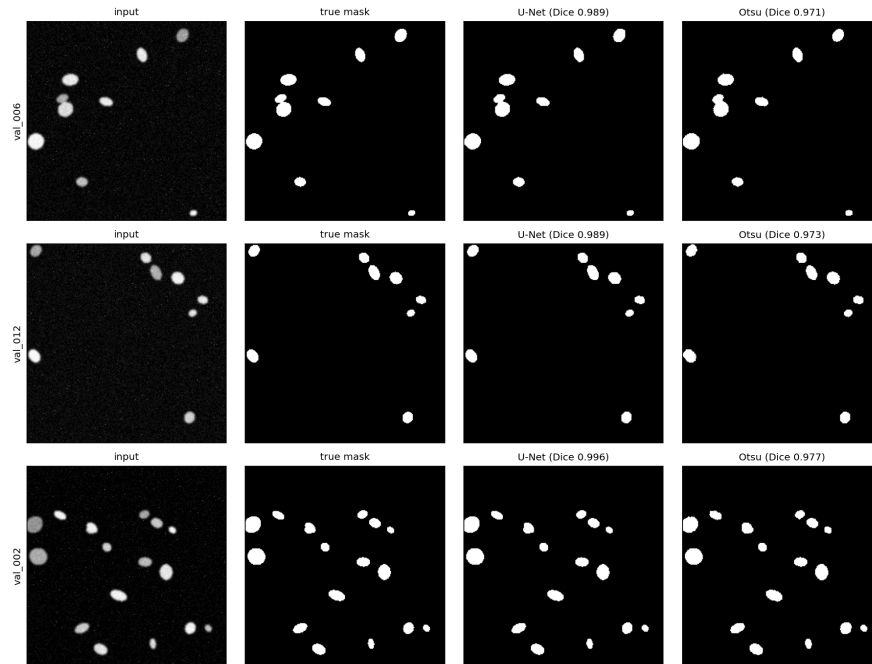


Fig. 4: Input / Ground Truth / U-Net Output / Otsu Output of the two lowest and highest performing validation image according to Dice.

5. Task 4 – Hybrid pipeline on unseen test images

Optimised prompt (same numbers-only contract, aimed at a technician reviewing the run)

You are the reporting phase of an audit trailable process. You receive some properties from a U-Net mask using regionprops. You have not seen the image. Provide two pieces of output: PART 1: One JSON object consisting of the following parameters: image_id, n_objects, mean_area (identical to input), density_class, quality_flag (using the same vocabularies as in Task 2). PART 2: One paragraph (3-4 sentences) addressed to a technician describing number of objects, average size/shape and variability, and quality flag if relevant.

Test Image Record Pipeline: U-Net prediction → RegionProps features → valid JSON record → narrative, aggregated in pipeline_records.csv (12 rows). Test image mean Dice 0.9918 / IoU 0.9838; quality flag “ok” for all 12 images. Example test image record (test_000, actual Ollama response):

{"image_id": "test_000", "n_objects": 8, "mean_area": 191.8, "density_class": "sparse", "quality_flag": "ok", "dice_vs_gt": 0.996} — narrative: “The image contains 8 objects with an average area of approximately 192 pixels... mostly circular with an average circularity of 0.984 and an average solidity of 0.969.” While test image scores show very high Dice score pixel accuracy, the predicted number of objects tends to be lower than the actual number (5.75 mean absolute error, -5.75 bias, e.g. test_004: 62 predicted vs 75 real) – even a perfect (more than 99% pixel-wise accurate) binary mask

5.1 Extension: robustness under corruption

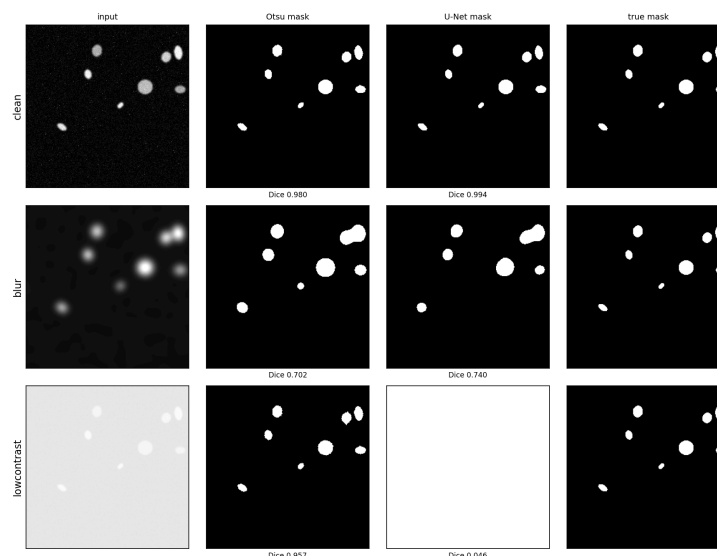


Fig. 5: test_000 clean / blurred / low contrast, using Otsu and U-Net masks.

Heavy blur compromises both approaches similarly (Otsu 0.702, U-Net 0.740 — focus_score decreases from 2.32 to 0.017, clear indicator on the image quality level, before calculating masks). But low contrast gives the most interesting results: Otsu is stable (0.957; it always uses the relative threshold, which adjusts to the current brightness level), whereas U-Net gets to a dismal 0.046 (predicts the whole image as foreground; it learnt an absolute brightness-based decision line from training examples and failed to

generalize it to another brightness level). Corruption becomes detectable on the pixel statistics level (contrast_score decreases from 0.21 to 0.03) — even before applying regionprops, segmentation or getting any value from the LLM.

6. Discussion

Q1. Which is more useful and which is more reliable, VLM description or numbers-first description?

Numbers-first description (Task 2) is more reliable since each number in its JSON structure was taken from deterministic measurement, and thus it cannot give wrong data about counts and sizes; its only flexibility is an item from a limited set of labels and some prose. VLM description (Task 1) is more useful for getting a quick impression of the field (its color, arrangement, and texture), but this is exactly where hallucination can show up: the naive prompt has generated the terms “tissue sample” and “stages of cell division” out of thin air, while the tissue_type from the structured prompt has varied twice among three identical runs.

Q2 Was U-Net better than Otsu? Provide one case for each method.

Yes, on all validation and test images free from artifacts (Dice mean +0.0186). Best case for U-Net: val_005 (clusters, 48 objects), where U-Net achieves 0.9922 Dice vs Otsu 0.9606 — U-Net can differentiate touching cells, whereas a fixed threshold cannot. Best case for Otsu: the robust version of test_000 to global illumination changes, Otsu 0.957 vs U-Net 0.046 — the latter fails because of an unseen shift of global illumination, which affects its decision

Q3. Dice Score/IoU - what is their meaning, where does the model fail?

Valid Dice 0.9927 / IoU 0.9856; Test Dice 0.9918 / IoU 0.9838 (BCE+Dice). Both Dice and IoU evaluate pixel-wise agreement between the masks (Dice is more weighted on the intersection part, while IoU is more strict), which means that the border of the mask is practically right everywhere. The misclassifications done by the model are not visible at the pixel level — they can be seen only one step higher, at the object count: the mask of the dense images is also >99% correct, but multiple touching nuclei become one connected piece, and regionprops detects too few of them (MAE for counts is 5.75, all the bias is negative). The images from Fig. 4 demonstrate that the worst cases still look quite good compared to the truth.

Q4. Where can the LLM hallucinate, and what makes this problem less likely?

Two spots: in the VLM step, the image generation could introduce visual or clinical information that is not in the pixels (blocked somewhat by the “uncertain” directive in the prompt, but not completely prevented — see the repeats table for label inconsistency under the constraints); and in the numbers-first steps, a measurement might be restated incorrectly or made up. The way to address the second problem almost completely is by validation: each number/vocabulary field in the JSON schema gets over-written by the computed value following the API call, and any discrepancy is logged in corrections. That’s how the JSON record, not the paragraph, becomes the “source of truth,” because the record is guaranteed to match the pixels, irrespective of the information provided by the model; whereas the paragraph is always just a readable explanation of numbers that have been validated already.

Q5. Would you trust this system in any way clinically? What one thing would improve it the most?

Nothing in this pipeline should be taken at face value for making any kind of clinical decision. This is true both for the synthetic dataset (perfect ground truth, one modality, no variation or variability in biology or staining, and no real patient population) as well as the small and minimally-trained U-Net that uses only 80 images in its training phase, and the instance-counting step, which is susceptible to failure based on the robustness study (where connected-components fails entirely outside of the training brightness range). The auditable JSON/CSV data trail is the one element here that truly stands out and is genuinely helpful in retrospect when trying to identify problems. The one step that could make the biggest improvement in terms of increasing trustworthiness would be to actually train the U-Net using instance segmentation (such as a learned watershed energy map or an instance-aware loss function).

References

- Ronneberger, O., Fischer, P. and Brox, T. (2015) 'U-Net: Convolutional Networks for Biomedical Image Segmentation', MICCAI 2015, pp. 234–241.
- Otsu, N. (1979) 'A Threshold Selection Method from Gray-Level Histograms', IEEE Transactions on Systems, Man, and Cybernetics, 9(1), pp. 62–66.
- van der Walt, S. et al. (2014) 'scikit-image: image processing in Python', PeerJ, 2:e453.
- Ji, Z. et al. (2023) 'Survey of Hallucination in Natural Language Generation', ACM Computing Surveys